corresponding docker-compose down command stops any running containers and removes them along with the app's dedicated network and volumes.

This is useful when you're done with a project and want to free up space that its resources were using. If you just want to remove the app's containers, there's the docker-compose rm command for this purpose.

Pruning: Freeing Up Unused Resources



Since we're talking about freeing up resources, there are some other commands that help us with this aim.

As we change our Dockerfile and rebuild images, some images inevitably will no longer be needed or used, yet sit there taking up precious disk space. These are known as *dangling* images; they can be removed with the docker image prune[5] command.

There's an entire family of prune commands to free up other unused resources (for example, docker container prune).[6] There's even a single command to free up all resources in one go:[7]

```
$ docker system prune
```

Quick Recap

We've introduced a powerful new tool into our arsenal: Docker Compose. It really is a one-stop-shop command for developing our app with Docker.

Let's review what we covered. In this chapter:

1. We introduced the docker-compose.yml and its format.
2. We created our own docker-compose.yml for our Rails app, including a locally mounted volume to allow live editing of local files.
3. We saw how to spin up our entire app and start the Rails server by using the command:

   ```
   $ docker-compose up
   ```

4. We learned various commands for managing our app with Compose:

   - List running containers

     ```
     $ docker-compose ps
     ```

5. https://docs.docker.com/engine/reference/commandline/image_prune/
6. https://docs.docker.com/engine/reference/commandline/container_prune/
7. https://docs.docker.com/engine/reference/commandline/system_prune/